# Ausführliche Lösungen Kompressionsverfahren

**Zum geführten Leitprogramm für 70 Minuten nach einer Einführung von etwa 20 Minuten.**

Dieses Dokument folgt genau den Experimenten im Arbeitsblatt. Vergleiche deine Lösung erst nach einer Station. Andere Formulierungen sind richtig, wenn sie dieselbe Bedeutung haben. Bei Huffman können verschiedene richtige Bäume entstehen.

## Vorbereitung und Stopp 0

Ein selbst erstellter Arbeitsordner und Notepad reichen für den Pflichtweg. Die Texte werden von den Lernenden erstellt. Die Bild-, Ton- und Metadatendateien aus DL05 sind hier nicht erforderlich. Für den Baum braucht es eine freie Zeichenfläche und zwei Stifte.

Die vereinbarte RLE-Regel lautet Anzahl plus Zeichen. 4B steht für genau BBBB. Einzelne Zeichen erhalten ebenfalls eine Anzahl, zum Beispiel 1A. Alle drei untersuchten Grundverfahren sind verlustfrei.

# Lösung 1 RLE codieren und decodieren

## 1.1 Gruppen und Code

Die Gruppen lauten WWWWWW, BBB und WW. Der Buchstabe wechselt erst nach sechs W und nach drei B. Die letzten beiden W bilden einen neuen Lauf; sie stehen nicht direkt neben den ersten sechs W.

| Gruppe | Anzahl | Darstellung |
|---|---|---|
| WWWWWW | 6 | 6W |
| BBB | 3 | 3B |
| WW | 2 | 2W |

Der vollständige RLE-Code ist 6W3B2W. Die Reihenfolge der Gruppen bleibt erhalten.

## 1.2 Sichtbare Längen

Das Original besitzt 6 + 3 + 2 = 11 Zeichen. 6W3B2W besitzt sechs sichtbare Zeichen: drei Ziffern und drei Buchstaben. Der vereinfachte Code ist damit um fünf Zeichen kürzer.

## 1.3 Zurückgelesener Text

6W wird zu WWWWWW, 3B zu BBB und 2W zu WW. Zusammengesetzt entsteht WWWWWWBBBWW. Alle elf Zeichen stimmen in Anzahl und Reihenfolge mit dem Original überein.

## 1.4 Verlustfreiheit

Ja, der Text ist genau gleich. Mit der vereinbarten Regel kann RLE die ursprünglichen Zeichen vollständig wiederherstellen. Deshalb ist RLE verlustfrei. Die Datei RLE_kontrolle.txt enthält den zurückgelesenen Text, nicht bloss eine Kopie des Originalcodes.

## 1.5 Weitere Beispiele

AAABCCCCCCDD enthält AAA, B, CCCCCC und DD. Daraus wird 3A1B6C2D. Wichtig ist 1B: Auch ein einzelnes B erhält eine Anzahl.
2K5L1M wird zu KK, LLLLL und M. Zusammengesetzt lautet die Folge KKLLLLLM. Es sind acht Zeichen.

Hilfe zur Korrektur: Gezählt werden hier nur sichtbare Zeichen. Eine tatsächliche Dateigrösse kann durch Zeilenumbrüche oder zusätzliche Codierungsangaben abweichen. Das ist kein Fehler in der RLE-Regel.

# Lösung 2 Expansion erkennen

### Vollständige Tabelle

| Original | Vorher | RLE-Code | Nachher |
|---|---|---|---|
| WWWWWWBBBWW | 11 | 6W3B2W | 6 |
| AAABCCCCCCDD | 12 | 3A1B6C2D | 8 |
| ABCDEFGH | 8 | 1A1B1C1D1E1F1G1H | 16 |

### 2.1 Welche Folge wird grösser

ABCDEFGH wird von acht auf sechzehn sichtbare Zeichen vergrössert. Es gibt keinen langen Lauf. Jeder Buchstabe benötigt im Code zusätzlich eine Ziffer 1. Der Code ist dadurch doppelt so lang wie das Original. Diese Vergrösserung heisst Expansion.

### 2.2 ABABABABABAB

Unsere RLE-Regel ist dafür ungeeignet. Nach jeder Stelle wechselt der Buchstabe. Es entstehen zwölf Läufe der Länge 1. Der Code lautet 1A1B1A1B1A1B1A1B1A1B1A1B. Er besitzt 24 sichtbare Zeichen statt zwölf.

Die Folge besitzt trotzdem ein Muster: Die Gruppe AB kommt sechsmal vor. Dieses Gruppenmuster eignet sich für ein Wörterbuchverfahren. RLE betrachtet in dieser Variante nur gleiche direkte Nachbarn.

### Was bedeutet verlustfrei

Auch die längere RLE-Darstellung lässt sich genau zurücklesen. Verlustfrei sagt etwas darüber aus, ob Informationen erhalten bleiben. Es sagt nicht, dass jede Eingabe nach dem Codieren kleiner wird.

# Lösung 3 Huffman Codes verwenden

### 3.1 BAD codieren

Aus der vorgegebenen Tabelle folgt B = 10, A = 0 und D = 111. Mit Trennstrichen lautet die Darstellung 10 | 0 | 111. Ohne Trennzeichen entsteht 100111. Die Codelängen sind 2 + 1 + 3 = 6 Bits.

### 3.2 Bitstrom gliedern

01110100 wird so gegliedert: 0 | 111 | 0 | 10 | 0. Lies stets ab der aktuellen Stelle, bis ein vollständiger Code erreicht ist. Danach beginnt die Suche wieder mit dem nächsten Bit.

| Gelesene Bits | Buchstabe | Noch ungelesene Bits |
|---|---|---|
| 0 | A | 1110100 |
| 111 | D | 0100 |
| 0 | A | 100 |
| 10 | B | 0 |
| 0 | A | keine |

### 3.3 Zurückgelesener Text

Die Folge lautet ADABA. Es bleiben keine Bits übrig. Beim Decodieren darf man nicht grundsätzlich zwei Bits auf einmal lesen, weil die Codes unterschiedlich lang sind.

### 3.4 Bei 0 sofort A

0 ist bereits ein vollständiger Code. Kein anderer Code der Tabelle beginnt mit 0: Die Codes 10, 110 und 111 beginnen alle mit 1. Darum ist eindeutig, dass 0 zu A gehört. Kein vollständiger Code ist der Anfang eines anderen; das nennt man präfixfrei.

Alltagsvergleich: Du folgst auf einem Wegweiser zuerst einer Richtung und danach gegebenenfalls weiteren Richtungen. Wenn du beim Ziel angekommen bist, beginnst du für das nächste Zeichen wieder am Anfang. Im Huffman-Baum ist ein Buchstabe ein solches Ziel, also ein Blatt.

# Lösung 4 Den Huffman Baum erstellen

### Häufigkeiten und Verbindungen

| Zeichen | E | T | A | N |
|---|---|---|---|---|
| Häufigkeit | 5 | 2 | 1 | 1 |

Die Summe ist 5 + 2 + 1 + 1 = 9. Zuerst sind A und N mit jeweils 1 am kleinsten. Verbinde A:1 + N:1 = 2. Frei sind jetzt dieser neue Knoten:2, T:2 und E:5.
Verbinde T:2 + Knoten:2 = 4. A und N werden nicht erneut einzeln ausgewählt. Frei sind jetzt Knoten:4 und E:5.
Verbinde E:5 + Knoten:4 = 9. Dieser letzte Knoten ist die Wurzel, also der Anfang aller Codewege.

### Eine mögliche Lösung

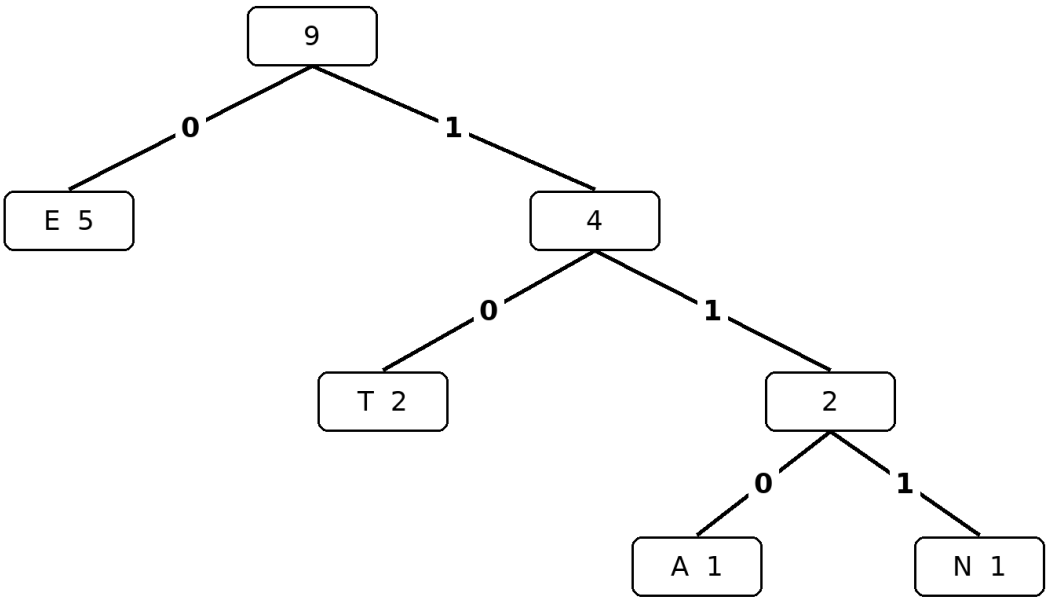



Abbildung 1  Huffman-Baum für EEEEETTAN. An den Ästen stehen die Bits, in den Knoten die Häufigkeiten.

Ein innerer Knoten fasst seine beiden Kinder zusammen. Ein Blatt enthält einen Buchstaben. Der Code wird von der Wurzel zum Blatt gelesen. Im gezeigten Baum sind die linken Äste mit 0 und die rechten mit 1 beschriftet.

Andere Anordnungen sind richtig, wenn in jedem Schritt die beiden kleinsten freien Häufigkeiten verbunden werden. Werden Teilbäume links und rechts vertauscht, ändern sich Codes, aber nicht ihre Längen und die Gesamtbitzahl.

# Lösung 4 Codes und Bitzahlen

### Codes des gezeigten Baums

| Zeichen | Code | Häufigkeit | Bits je Code | Bits insgesamt |
|---|---|---|---|---|
| E | 0 | 5 | 1 | 5 × 1 = 5 |
| T | 10 | 2 | 2 | 2 × 2 = 4 |
| A | 110 | 1 | 3 | 1 × 3 = 3 |
| N | 111 | 1 | 3 | 1 × 3 = 3 |
| Total | | 9 | | 15 |

E liegt direkt neben der Wurzel und benötigt nur einen Ast, also ein Bit. T braucht zwei Äste. A und N benötigen drei Äste. Das häufige E erhält dadurch den kürzesten Code.

### 4.2 TAN codieren

Mit dem gezeigten Baum gilt T = 10, A = 110 und N = 111. Mit Trennstrichen entsteht 10 | 110 | 111. Der Bitstrom lautet 10110111. Er benötigt 2 + 3 + 3 = 8 Bits. Ein anderer richtiger Baum kann einen anderen Bitstrom liefern. Die Lösung muss dann zu den eigenen Codes passen.

### 4.3 bis 4.4 Bitzahlen vergleichen

Feste Zuordnung: 9 Zeichen × 2 Bits = 18 Bits.
Huffman-Code: 5 × 1 + 2 × 2 + 1 × 3 + 1 × 3 = 15 Bits.
Einsparung für den Text allein: 18 − 15 = 3 Bits.

Kontrolle: EEEEETTAN wird mit diesen Codes zu 0 | 0 | 0 | 0 | 0 | 10 | 10 | 110 | 111. Ohne Trennzeichen lautet der vollständige Bitstrom 000001010110111. Er besitzt 15 Bits.

### 4.5 Kürzester Code

E kommt fünfmal vor und damit am häufigsten. Sein Code besitzt ein Bit. Ein kurzer Code für ein häufiges Zeichen spart immer wieder Platz. Ein längerer Code für ein seltenes Zeichen wird dagegen nur selten benötigt.

### Grenze dieses Vergleichs

Die 15 und 18 Bits zählen nur die Nutzdaten des Beispiels. Echte Dateien benötigen zusätzliche Angaben, etwa die Codezuordnung und die Länge der Daten. Ausserdem werden Dateien in ganzen Bytes gespeichert. Bei so wenig Text können diese Zusatzangaben die Einsparung übersteigen. Eine in Notepad getippte Bitfolge ist kein gepackter Huffman-Bitstrom.

# Lösung 5 Das LZW Wörterbuch

Startwörterbuch: A = 1 und B = 2. Neue Einträge erhalten der Reihe nach die Codes 3, 4 und 5. Wichtig: Das nächste Zeichen wird zwar für den neuen Eintrag verwendet, bleibt aber zunächst im Resttext.

| Schritt | Rest vorher | Gruppe | Ausgabe | Neuer Eintrag | Rest danach |
|---|---|---|---|---|---|
| 1 | ABABABA | A | 1 | AB = 3 | BABABA |
| 2 | BABABA | B | 2 | BA = 4 | ABABA |
| 3 | ABABA | AB | 3 | ABA = 5 | ABA |
| 4 | ABA | ABA | 5 | kein neuer Eintrag | leer |

### Warum diese vier Schritte stimmen

Schritt 1: AB ist noch unbekannt. A ist bekannt und wird als 1 ausgegeben. A plus das folgende B wird als AB = 3 eingetragen. Nur A wird aus dem Rest entfernt.
Schritt 2: BA ist noch unbekannt. B wird als 2 ausgegeben. B plus das folgende A ergibt BA = 4. Nur B wird entfernt.
Schritt 3: AB ist bekannt, ABA noch nicht. Daher ist AB die längste bekannte Gruppe. Ihre Ausgabe ist 3. AB plus das folgende A ergibt ABA = 5. Nur AB wird entfernt.
Schritt 4: Der Rest lautet ABA. Dieser Eintrag ist nun bekannt und wird als 5 ausgegeben. Danach ist kein Rest mehr vorhanden. Es gibt auch keinen nächsten Buchstaben für einen neuen Eintrag.

### 5.1 Ausgabecodes

Die Codes lauten 1, 2, 3, 5. Die verwendeten Gruppen sind A, B, AB und ABA. Zusammengesetzt ergeben sie die sieben ursprünglichen Zeichen ABABABA.

### 5.2 Wörterbuch beim Empfänger

Der Empfänger beginnt mit demselben Startwörterbuch. Mit denselben Decodierregeln kann er die neuen Einträge in derselben Reihenfolge aufbauen. Das ganze neu aufgebaute Wörterbuch muss deshalb nicht separat mitgesendet werden. Beide Seiten brauchen aber dieselben Regeln und Informationen über das Startwörterbuch.

Lehrpersonenhinweis: Die vollständige LZW-Decodierung wird hier nicht verlangt. Beim letzten Code 5 tritt im Standardalgorithmus der Fall auf, dass ein Eintrag beim Decoder gerade erst entsteht. Ein blosses Nachschlagen aller Codes im anfänglichen Wörterbuch genügt nicht.

Die vier sichtbaren Nummern beweisen noch keine kleinere Datei. In einem echten LZW-Format brauchen Codes eine festgelegte Bitbreite und weitere Formatangaben.

# Lösung 6 Auswahl und Kurzcheck

### Zuordnung der Grundideen

| Daten | Grundidee | Begründung |
|---|---|---|
| AAAAAAAAAAAAAAAAAAA | RLE | A kommt zwanzigmal direkt nacheinander vor. |
| Langer Text mit vielen E und wenigen Q | Huffman | Häufige Zeichen können kurze Codes erhalten. |
| M114-TEST-M114-TEST-M114-TEST | LZ/LZW | Eine ganze Buchstabengruppe wiederholt sich. |

Die Tabelle ordnet passende Grundideen zu. Sie behauptet nicht, dass für diese Daten nur ein Verfahren möglich ist. Kompressionsprogramme kombinieren häufig verschiedene Verfahren.

### 6.1 Unterschied bei ABABAB

RLE sieht sechs Läufe der Länge 1. LZW kann dagegen die wiederkehrende Gruppe AB in sein Wörterbuch aufnehmen und später durch einen Code ersetzen. Direkt gleiche Zeichen und wiederkehrende Gruppen sind unterschiedliche Muster.

### 6.2 Was speichert RLE

RLE speichert die Anzahl und das Zeichen: BBBBBB wird zu 6B.

### 6.3 Welche Zeichen bekommen kurze Codes

Häufige Zeichen erhalten kurze Codes. Seltene Zeichen dürfen längere Codes erhalten. In unserem Beispiel ist E das häufigste Zeichen und besitzt den kürzesten Code.

### 6.4 Was ersetzt LZW

LZW ersetzt bekannte Zeichenketten beziehungsweise Buchstabengruppen durch ihre Wörterbuchnummer. Ein Eintrag kann zunächst ein einzelner Buchstabe und später eine längere Gruppe sein.

### 6.5 Verlustfreiheit

Ja. RLE, Huffman und LZW sind verlustfrei, wenn nach den vereinbarten Regeln korrekt codiert und decodiert wird. Alle ursprünglichen Zeichen oder Bytes können vollständig wiederhergestellt werden. Es werden keine Informationen absichtlich entfernt.

# Lösungen zu den Zusatzaufgaben

## Zusatz A Binärfolge

Die Gruppen der Folge sind 000000, 111, 00000 und 11111111. Der Startwert ist 0. Die Lauflängen sind 6, 3, 5, 8. Beim Zurücklesen werden sechs Nullen, drei Einsen, fünf Nullen und acht Einsen geschrieben. Zusammen entsteht wieder 000000111000001111111 1.

Der Startwert ist nötig. Ohne ihn wäre nicht klar, ob die erste Gruppe Nullen oder Einsen enthält. Die Summe der Lauflängen ist 6 + 3 + 5 + 8 = 22 Bits.

| Pixelzeile | RLE-Code | Sichtbare Codelänge |
|---|---|---|
| W W W B B B B B | 3W5B | 4 |
| R R R R W W B B | 4R2W2B | 6 |
| W B W B W B W B | 1W1B1W1B1W1B1W1B | 16 |

Die dritte Zeile eignet sich am schlechtesten. Nach jedem Pixel wechselt die Farbe. Für acht Pixelwerte entstehen sechzehn sichtbare Zeichen im Übungscode. Die Leerzeichen in der Ausgangszeile waren nur zur Lesbarkeit eingefügt und zählen nicht als Pixel.

## Zusatz B ABABAB mit LZW

| Rest vorher | Gruppe | Ausgabe | Neuer Eintrag |
|---|---|---|---|
| ABABAB | A | 1 | AB = 3 |
| BABAB | B | 2 | BA = 4 |
| ABAB | AB | 3 | ABA = 5 |
| AB | AB | 3 | keiner, Ende |

Die Ausgabecodes lauten 1, 2, 3, 3. Der letzte Rest ist AB, nicht ABA. Deshalb lautet der letzte Code 3.

## Zusatz C ZIP Versuch

Die eigenen Messwerte sind richtig, wenn Grösse in Bytes für die Textdatei und für das gesamte Archiv abgelesen wurde. Die wiederholten Wörter und ganzen Zeilen bieten nutzbare Muster. Bei ausreichend vielen Zeilen wird ein übliches ZIP-Archiv meistens deutlich kleiner. Die genaue Grösse hängt unter anderem von Zeilenzahl, Zeilenumbrüchen, Dateinamen und ZIP-Programm ab.

Allein an der Dateigrösse lässt sich das interne Verfahren nicht sicher erkennen. Verschiedene Verfahren können ähnlich grosse Dateien erzeugen. Für eine sichere Aussage braucht es Angaben zum Format oder zur tatsächlich verwendeten Kompressionsmethode. Bei vielen ZIP-Archiven wird Deflate verwendet; es kombiniert eine LZ-Methode mit Huffman. Der Grössenvergleich allein beweist diesen Einsatz jedoch nicht.